An Analysis of K-nearest neighbors algorithm

K-nearest neighbors algorithm

K-nearest neighbors algorithm -- Part 1

Parameter selection The best choice of k depends upon the data; generally, larger values of k reduces effect of the noise on the classification, but make boundaries between classes less distinct. A good k can be selected by various heuristic techniques (see hyperparameter optimization). The special case where the class is predicted to be the class of the closest training sample (i.e. when k = 1) is called the nearest neighbor algorithm. The accuracy of the k-NN algorithm can be severely degraded by the presence of noisy or irrelevant features, or if the feature scales are not consistent with their importance. Much research effort has been put into selecting or scaling features to improve classification. A particularly popular approach is the use of evolutionary algorithms to optimize feature scaling. Another popular approach is to scale features by the mutual information of the training data with the training classes. In binary (two class) classification problems, it is helpful to choose k to be an odd number as this avoids tied votes. One popular way of choosing the empirically optimal k in this setting is via bootstrap method.

K-nearest neighbors algorithm -- Part 2

where R ∗ {\displaystyle R^{*}} is the Bayes error rate (which is the minimal error rate possible), R k N N {\displaystyle R_{kNN}} is the asymptotic k-NN error rate, and M is the number of classes in the problem. This bound is tight in the sense that both the lower and upper bounds are achievable by some distribution. For M = 2 {\displaystyle M=2} and as the Bayesian error rate R ∗ {\displaystyle R^{*}} approaches zero, this limit reduces to "not more than twice the Bayesian error rate".

K-nearest neighbors algorithm -- Part 3

R R (C n w n n) – R R (C Bayes) = (B 1 s n 2 + B 2 t n 2) { 1 + o (1) } , {\displaystyle {\mathcal {R}}_{\mathcal {R}}(C_{n}^{wnn})-{\mathcal {R}}_{\mathcal {R}}(C^{\text{Bayes}})=\left(B_{1}s_{n}^{2}+B_{2}t_{n}^{2}\right)\{1+o(1)\},}

K-nearest neighbors algorithm -- Part 4

The training examples are vectors in a multidimensional feature space, each with a class label. The training phase of the algorithm consists only of storing the feature vectors and class labels of the training samples. In the classification phase, k is a user-defined constant, and an unlabeled vector (a query or test point) is classified by assigning the label which is most frequent among the k training samples nearest to that query point.

K-nearest neighbors algorithm -- Part 5

Condensed Nearest Neighbor for data reduction Condensed nearest neighbor (CNN, the Hart algorithm) is an algorithm designed to reduce the data set for k-NN classification. It selects the set of prototypes U from the training data, such that 1NN with U can classify the examples almost as accurately as 1NN does with the whole data set.

K-nearest neighbors algorithm -- Part 6

In k-NN regression, also known as k-NN smoothing, the k-NN algorithm is used for estimating continuous variables. One such algorithm uses a weighted average of the k nearest neighbors, weighted by the inverse of their distance. This algorithm works as follows:

K-nearest neighbors algorithm -- Part 7

Scan all elements of X, looking for an element x whose nearest prototype from U has a different label than x. Remove x from X and add it to U Repeat the scan until no more prototypes are added to U. Use U instead of X for classification. The examples that are not prototypes are called "absorbed" points. It is efficient to scan the training examples in order of decreasing border ratio. The border ratio of a training example x is defined as

K-nearest neighbors algorithm -- Part 8

Feature extraction When the input data to an algorithm is too large to be processed and it is suspected to be redundant (e.g. the same measurement in both feet and meters) then the input data will be transformed into a reduced representation set of features (also named features vector). Transforming the input data into the set of features is called feature extraction. If the features extracted are carefully chosen it is expected that the features set will extract the relevant information from the input data in order to perform the desired task using this reduced representation instead of the full size input. Feature extraction is performed on raw data prior to applying k-NN algorithm on the transformed data in feature space. An example of a typical computer vision computation pipeline for face recognition using k-NN including feature extraction and dimension reduction pre-processing steps (usually implemented with OpenCV):

K-nearest neighbors algorithm -- Part 9

R R (C n k n n) – R R (C Bayes) = { B 1 1 k + B 2 (k n) 4 / d } { 1 + o (1) } , {\displaystyle {\mathcal {R}}_{\mathcal {R}}(C_{n}^{knn})-{\mathcal {R}}_{\mathcal {R}}(C^{\text{Bayes}})=\left\{B_{1}{\frac {1}{k}}+B_{2}\left({\frac {k}{n}}\right)^{4/d}\right\}\{1+o(1)\},}

K-nearest neighbors algorithm -- Part 10

Error rates There are many results on the error rate of the k nearest neighbour classifiers. The k-nearest neighbour classifier is strongly (that is for any joint distribution on (X , Y) {\displaystyle (X,Y)}) consistent provided k := k n {\displaystyle k:=k_{n}} diverges and k n / n {\displaystyle k_{n}/n} converges to zero as n → ∞ {\displaystyle n\to \infty } . Let C n k n n {\displaystyle C_{n}^{knn}} denote the k nearest neighbour classifier based on a training set of size n. Under certain regularity conditions, the excess risk yields the following asymptotic expansion

K-nearest neighbors algorithm -- Part 11

In statistics, the k-nearest neighbors algorithm (k-NN) is a non-parametric supervised learning method. It was first developed by Evelyn Fix and Joseph Hodges in 1951, and later expanded by Thomas Cover. In classification, a new example is assigned a label based on the labels of its k nearest training examples; in regression, the prediction is computed from the values of those neighbors. Most often, it is used for classification, as a k-NN classifier, the output of which is a class membership. An object is classified by a plurality vote of its neighbors, with the object being assigned to the class most common among its k nearest neighbors (k is a positive integer, typically small). If k = 1, then the object is simply assigned to the class of that single nearest neighbor. The k-NN algorithm can also be generalized for regression. In k-NN regression, also known as nearest neighbor smoothing, the output is the property value for the object. This value is the average of the values of k nearest neighbors. If k = 1, then the output is simply assigned to the value of that single nearest neighbor, also known as nearest neighbor interpolation. For both classification and regression, a useful technique can be to assign weights to the contributions of the neighbors, so that nearer neighbors contribute more to the average than distant ones. For example, a common weighting scheme consists of giving each neighbor a weight of 1/d, where d is the distance to the neighbor. The input consists of the k closest training examples in a data set. The neighbors are taken from a set of objects for which the class (for k-NN classification) or the object property value (for k-NN regression) is known. This can be thought of as the training set for the algorithm, though no explicit training step is required. A peculiarity (sometimes even a disadvantage) of the k-NN algorithm is its sensitivity to the local structure of the data. In k-NN classification the function is only approximated locally and all computation is deferred until function evaluation. Since this algorithm relies on distance, if the features represent different physical units or come in vastly different scales, then feature-wise normalizing of the training data can greatly improve its accuracy.